JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Interface TemporalAmount

**All Known Subinterfaces:**

ChronoPeriod

**All Known Implementing Classes:**

Duration, Period

---

`public interface` **`TemporalAmount`**

Framework-level interface defining an amount of time, such as "6 hours", "8 days" or "2 years and 3 months".

This is the base interface type for amounts of time. An amount is distinct from a date or time-of-day in that it is not tied to any specific point on the time-line.

The amount can be thought of as a `Map` of `TemporalUnit` to `long`, exposed via `getUnits()` and `get(TemporalUnit)`. A simple case might have a single unit-value pair, such as "6 hours". A more complex case may have multiple unit-value pairs, such as "7 years, 3 months and 5 days".

There are two common implementations. `Period` is a date-based implementation, storing years, months and days. `Duration` is a time-based implementation, storing seconds and nanoseconds, but providing some access using other duration based units such as minutes, hours and fixed 24-hour days.

This interface is a framework-level interface that should not be widely used in application code. Instead, applications should create and pass around instances of concrete types, such as `Period` and `Duration`.

**Implementation Requirements:**

This interface places no restrictions on the mutability of implementations, however immutability is strongly recommended.

**Since:**

1.8

## *Method Summary*

| All Methods | Instance Methods | |
|---|---|---|
| **Modifier and Type** | **Method** | **Description** |
| `Temporal` | **`addTo`**`(Temporal temporal)` | Adds to the specified temporal object. |
| `long` | **`get`**`(TemporalUnit unit)` | |

| | | |
|---|---|---|
| | | Returns the value of the requested unit. |
| List<TemporalUnit> | **getUnits**() | Returns the list of units uniquely defining the value of this TemporalAmount. |
| Temporal | **subtractFrom**(Temporal temporal) | Subtracts this object from the specified temporal object. |

## *Method Details*

### get

`long get(TemporalUnit unit)`

Returns the value of the requested unit. The units returned from `getUnits()` uniquely define the value of the `TemporalAmount`. A value must be returned for each unit listed in `getUnits`.

**Implementation Requirements:**

Implementations may declare support for units not listed by `getUnits()`. Typically, the implementation would define additional units as conversions for the convenience of developers.

**Parameters:**

`unit` - the `TemporalUnit` for which to return the value

**Returns:**

the long value of the unit

**Throws:**

`DateTimeException` - if a value for the unit cannot be obtained

`UnsupportedTemporalTypeException` - if the `unit` is not supported

### getUnits

`List<TemporalUnit> getUnits()`

Returns the list of units uniquely defining the value of this TemporalAmount. The list of `TemporalUnits` is defined by the implementation class. The list is a snapshot of the units at the time `getUnits` is called and is not mutable. The units are ordered from longest duration to the shortest duration of the unit.

**Implementation Requirements:**

The list of units completely and uniquely represents the state of the object without omissions, overlaps or duplication. The units are in order from longest duration to shortest.

**Returns:**

the List of `TemporalUnits`; not null

## addTo

`Temporal addTo(Temporal temporal)`

Adds to the specified temporal object.

Adds the amount to the specified temporal object using the logic encapsulated in the implementing class.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.plus(TemporalAmount)`:

```
// These two lines are equivalent, but the second approach is recommended
dateTime = amount.addTo(dateTime);
dateTime = dateTime.plus(amount);
```

It is recommended to use the second approach, `plus(amount)`, as it is a lot clearer to read in code.

**Implementation Requirements:**

The implementation must take the input object and add to it. The implementation defines the logic of the addition and is responsible for documenting that logic. It may use any method on `Temporal` to query the temporal object and perform the addition. The returned object must have the same observable type as the input object

The input object must not be altered. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable temporal objects.

The input temporal object may be in a calendar system other than ISO. Implementations may choose to document compatibility with other calendar systems, or reject non-ISO temporal objects by `querying the chronology`.

This method may be called from multiple threads in parallel. It must be thread-safe when invoked.

**Parameters:**

`temporal` - the temporal object to add the amount to, not null

**Returns:**

an object of the same observable type with the addition made, not null

**Throws:**

`DateTimeException` - if unable to add

`ArithmeticException` - if numeric overflow occurs

## subtractFrom

`Temporal subtractFrom(Temporal temporal)`

Subtracts this object from the specified temporal object.

Subtracts the amount from the specified temporal object using the logic encapsulated in the implementing class.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.minus(TemporalAmount)`:

```
// these two lines are equivalent, but the second approach is recommended
dateTime = amount.subtractFrom(dateTime);
dateTime = dateTime.minus(amount);
```

It is recommended to use the second approach, `minus(amount)`, as it is a lot clearer to read in code.

**Implementation Requirements:**

The implementation must take the input object and subtract from it. The implementation defines the logic of the subtraction and is responsible for documenting that logic. It may use any method on `Temporal` to query the temporal object and perform the subtraction. The returned object must have the same observable type as the input object

The input object must not be altered. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable temporal objects.

The input temporal object may be in a calendar system other than ISO. Implementations may choose to document compatibility with other calendar systems, or reject non-ISO temporal objects by `querying the chronology`.

This method may be called from multiple threads in parallel. It must be thread-safe when invoked.

**Parameters:**

`temporal` - the temporal object to subtract the amount from, not null

**Returns:**

an object of the same observable type with the subtraction made, not null

**Throws:**

`DateTimeException` - if unable to subtract

`ArithmeticException` - if numeric overflow occurs

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.